Week 4 Lab: PI Motor-Speed Control

ENGR 3303 – Controls Engineering
Fall 2026

Main goal. Implement PI motor-speed control, compare it with P control, tune the integral gain K_i, observe integral windup behavior, and select a final controller using measured data.

What Changes from Week 3?

Week 3 used proportional control:

$$u = K_p e.$$

Week 4 adds integral action:

$$\boxed{u = K_p e + K_i I}$$

where

$$I[k] = I[k-1] + e[k]T_s.$$

The Arduino will send the following columns to Python:

time, reference RPM, RPM, error, PWM, integral

Hardware and Software

Use the same hardware from Weeks 1–3:

- Arduino Uno or compatible board;
- L298N motor-driver module;
- 12 V encoder gearmotor;
- external motor power supply;
- Arduino IDE;
- Python with pyserial, pandas, NumPy, and Matplotlib.

Install Python packages:

```
pip install pyserial pandas numpy matplotlib scipy
```

Safety

- Turn motor power OFF before changing wiring.
- Do not touch the spinning wheel.
- Stop testing if the motor becomes rough, oscillatory, or repeatedly saturates for too long.
- Keep Arduino GND and L298N GND connected.
- Do not increase K_i aggressively.

Pin Assignment

Use the same wiring as before.

Function	Arduino pin	Connection
Motor PWM	D9	L298N ENA
Motor direction 1	D7	L298N IN1
Motor direction 2	D8	L298N IN2
Encoder A	D2	yellow encoder lead
Encoder B	D3	green encoder lead
Encoder VCC	5 V	blue encoder lead
Common ground	GND	black encoder lead and L298N GND

Course encoder calibration: Use `COUNTS_PER_OUTPUT_REV = 494.0` for Encoder A counted with `RISING` edges only. Recalibrate only if the motor or encoder-counting method changes.

Lab 4A – Implement PI Control

Suggested time: 75 minutes

Part 1 – Choose Reference and Baseline Gain

Use the same reference speed as Week 3 if possible.

$$\boxed{r = _____ \text{ RPM}}$$

Use the selected K_p from Week 3.

$$\boxed{K_p = _____}$$

For the first test, use

$$\boxed{K_i = 0}$$

This creates a baseline P-only test.

Part 2 – Upload the PI Controller Code

Open:

`week4_pi_controller.ino`

Before uploading, verify:

- `REFERENCE_RPM`;
- `KP`;
- `KI`;
- `COUNTS_PER_OUTPUT_REV`;
- `INTEGRAL_MIN` and `INTEGRAL_MAX`.

Part 3 – Record the Baseline Test

Close Arduino Serial Monitor.

Open and run:

`week4_record_pi_control.py`

Set a filename such as

`PI_Ki0.csv`.

This file is the P-only baseline.

> **Lab 4A checkoff:** Show the instructor one plot of reference RPM and measured RPM, one PWM plot, and one integral-state plot.

Part 4 – Add a Small Integral Gain

Now choose a small value of K_i, for example:

$$K_i = 0.1 \quad \text{or} \quad 0.2.$$

Use instructor guidance if your motor behaves differently.

Record:

$$\boxed{K_i = _____}$$

Upload the code and run Python again.
Save as:

`PI_Ki0p2.csv`

or a similar descriptive name.

Part 5 – First Comparison

Complete:

K_i	Final RPM	Error	Max PWM	Observation
0				
small K_i				

Lab 4B – Tune K_i

Suggested time: 75 minutes

Part 1 – Test Three Integral Gains

Keep K_p fixed.
Test three values of K_i, including $K_i = 0$.
Suggested starting values:

$$\boxed{K_i = 0, \quad 0.2, \quad 0.6}$$

Adjust with instructor approval.
For each test:

1. change `KI` in Arduino;
2. upload the code;
3. run the Python recorder;
4. save a different CSV file;
5. inspect the plots before increasing K_i.

> If the response oscillates strongly, overshoots too much, or the motor sounds rough, stop increasing K_i.

Part 2 – Compare the Tests

Run:

`week4_compare_ki.py`

This script creates:

- RPM comparison plot;
- PWM comparison plot;
- integral-state comparison plot;
- summary table.

Part 3 – Calculate Overshoot

For each test:

$$\text{Overshoot} = y_{\max} - r.$$

If the speed never exceeds the reference, report zero overshoot.

Part 4 – Complete the Table

K_i	Final RPM	Error	Overshoot	Max PWM	Observation

Part 5 – Select a PI Controller

Choose a final pair:

$$\boxed{K_p = _____, \qquad K_i = _____}$$

Justify your choice using:

- steady-state error;
- overshoot;
- settling behavior;
- PWM saturation;
- integral-state behavior;
- smoothness.

Questions to Submit

Answer briefly.

1. What reference RPM did your team use?
2. What K_p did you keep fixed?
3. What K_i values did you test?
4. How did adding integral action affect steady-state error?
5. Which K_i gave the smallest steady-state error?
6. Which K_i gave the smoothest response?
7. Did any test show overshoot?
8. Did any test show long PWM saturation?
9. What is integral windup?
10. How does integral clamping help?
11. Which K_p, K_i pair did your team select and why?

What to Submit

One submission per team:

- CSV data for all PI tests;
- RPM comparison plot;
- PWM comparison plot;
- integral-state comparison plot;
- completed comparison table;
- selected K_p, K_i with justification;
- answers to Questions 1–11;
- Arduino code and Python code.

End-of-Lab Shutdown

Before leaving:

1. set PWM to zero;
2. turn OFF motor power;
3. save all CSV, Arduino, and Python files;
4. disconnect USB after motor power is off;
5. return hardware in an organized condition.